```
    bool try_pop(data_type& res)      <-3
    {
        std::lock_guard<std::mutex> lock(the_mutex);
        if(the_queue.empty())
        {
            return false;
        }

        res=std::move(the_queue.front());
        the_queue.pop_front();
        return true;
    }

    bool try_steal(data_type& res)      <-4
    {
        std::lock_guard<std::mutex> lock(the_mutex);
        if(the_queue.empty())
        {
            return false;
        }

        res=std::move(the_queue.back());
        the_queue.pop_back();
        return true;
    }
};
```

This queue is a simple wrapper around a `std::deque<function_wrapper>` ❶ that protects all accesses with a mutex lock. Both `push()` ❷ and `try_pop()` ❸ work on the front of the queue, while `try_steal()` ❹ works on the back.

This actually means that this "queue" is a last-in-first-out stack for its own thread; the task most recently pushed on is the first one off again. This can help improve performance from a cache perspective, because the data related to that task is more likely to still be in the cache than the data related to a task pushed on the queue previously. Also, it maps nicely to algorithms such as Quicksort. In the previous implementation, each call to `do_sort()` pushes one item on the stack and then waits for it. By processing the most recent item first, you ensure that the chunk needed for the current call to complete is processed before the chunks needed for the other branches, thus reducing the number of active tasks and the total stack